When someone comments **"pricing"** on any Instagram post:

1. Reply to the comment with **"Sent you a DM!"**
2. Send the user a DM asking for their email address.
3. Wait for the user to respond.
4. Once an email address is received, send them another DM containing a link.

I will go with a simple solution, not a generic one, because I have information only about this requirements, if anything changes in the future, the solution can be extended.

I will call my automation service: **PricingAutomation**

PricingAutomation is part of the ***MainSystem*** that monitor Instagram posts and their comments, DMs, using a queue system for notification, so my service will subscribe to the specific queue where the posts comments are receiving(I'm expecting the comments to be for specific posts) and to the queue for the DMs.

The ***MainSystem*** should have a kind of DeadLetterQueue where message queues that were not properly consumed by the **PricingAutomation** and other consumers are stored , for debugging and retry mechanisms.

Models - tables in PostgreSQL
***pricing_automation_setup* table - this is the table where the customer save the automation he creates from the UI**
id - PK for the table, autoincrement, number
user_id - ***MainSystem*** user
instagram_user_id - maybe the user has multiple instagram accounts.
comment_keyword - in our case is the 'pricing' keyword
comment_reply_message - message to send as a response on the instagram user comment
dm_message - message to send it as a DM

***pricing_automation* table - this is where the automation mechanism keeps its data**
id - PK for the table, autoincrement, number
user_id - ***MainSystem*** user
instagram_post_id - the post I follow the comments
instagram_comment_id - the comment that was sent to the post_id
instagram_user_id - user who posted the comment, is the id from Instagram
instagram_user_email - email of the user who posted the comment
automation_step - at which step is currently and has values [INITIAL, DM_ASKING_EMAIL, REPLY_COMMENT, DM_WITH_LINK], by default is INITIAL
nr_of_retries - how many retries in case of a failure
comment - a column to be used for any kind of information that helps for the debug, or storing an error message & stack; can have a JSON structure
created_at - creation date, for audit purpose (both fields for the date can be used for let's say you want to delete/archive data older than X weeks/months) - this can help to lower the content of the table and faster queries
updated_at - updated date, for audit purpose

Add unique constraint on user_id, instagram_post_id, instagram_comment_id, instagram_user_id
- I don't want to process the same message multiple times

Add composed index on (user_id, post_id, comment_id, instagram_user_id)
- Used for a faster search for a specific comment

Add an index to automation_step
- Using it to check unprocessed comments

When **PricingAutomation** receives a new comment will enter in a pipeline mode

CRUD for pricing_automation_setup table

- A POST endpoint for Create
- A PUT endpoint for Update
- A DELETE endpoint for deletion

Handler who is listening to the queue for the posts comments

Step1:

- Check if the customer already posted a comment with **pricing** (query using the composed index, only messages with **pricing** are saved)
- If the user already posted a comment, do nothing
- If is the first comment of the user, go to Step2

Step2 - *programmed version:*

- Check if the message is not empty
- Lower case the entire message
- Check with a regexp or string utility if the message contains the desire keyword
    - I don't understand the need of having * in the ****"pricing"****, this is why I would chose to go with regexp
    - If I would need a deeper refinement, let's say the word **pricing** is in another language, I would use a translation library to detect the word
    - Another situation would be that the comment is not using the complete keyword **pricing**, in this case I would do a match of let's say 90% - I used ChatGPT to find a way for this and I get the name of a library: *fastest-levenshtein*
- If the comment is valid, save data to *pricing_automation table*
    - If the save is failing because of the unique constraint, log a warning message.
- AI based semantic matching can be introduced later or if is decided is needed

Step 3

- I would choose to first send the message to the user, then reply to its comment, to be sure the message was sent. I assume Instagram has an API for this. The following operations will be sequentially asynchronous
    - Send the DM to the user
    - Update *automation_step* with *DM_ASKING_EMAIL*
    - Reply to it's comment from the post
    - Update *automation_step* with *REPLY_COMMENT*
    - I would have a max(to be defined, maybe 5) retry mechanism for each of the operations from this step, each retry is before going to the next step and the *nr_of_retries* is reset to 0 before the second action. I will update table with each retry

Handler who is listening to the queue for the DMs:

Step 1:

- Check if I have a pricing post from the user of the DM(instagram_user_id): by using the composed index
    - If I do not have, I do nothing(or send a queue notification for other automation process)
    - If I have, then I check the DM for an valid email pattern: the same using regexp
- If I have a valid email address,
    - If the email is valid, I update the *pricing_automation* table
    - I send it in a *PRICING_EMAIL_SQS* queue with *pricing_automation.id*

PRICING_EMAIL_SQS handler

- Check the DM history for the message I want to send
    - If the message was already sent, I'll do nothing
    - If the message was not sent, I'll send it and update *automation_step* to *DM_WITH_LINK*

For this service I would do

- AI plan
- AI implementation

- Human review
- AI unit and integration tests
- Human review
- Code coverage >= 90%
- Code review by AI and Human
- DEV and PROD environments based on a single github main branch, using Tags for release on PROD

For this solution I would do more things:

- The service stays in a container in k8s for an easy way to multiply in case more resources needed
- The deploy can be automated via a github action
    - Install dependencies
    - Check typescript lint
    - Build the packed
    - Build the Docker Image
    - Upload to an Image Registry Repository
    - Use a CD tool like ArgoCD to deploy it on the cloud
- I would add logs with different levels: info, debug, error and send message to Slack/Teams for a faster way of knowing about issues
- I would add monitors like
    - Postgresql cpu/ram 70-80%

Additional note:

I designed the system myself, then asked ChatGPT for a review. I incorporated some suggestions regarding unique constraints in the *pricing_automation* table and clearer field naming.

I'm aware that the solution doesn't guarantee that a DM will be sent only once in certain failure scenarios, like failing to save the last status to postgresql, but I intentionally time-boxed the assessment to 2 hours to focus on a simple, practical solution.